

AI-Assisted Software Development Workflow

*A Human-Centered Framework for Thinking, Building,
and Maintaining Software in the Age of AI*

Engineering version 2.1

20/May/2026

AI accelerates development. Responsibility never leaves the engineer.

Abstract

Software engineering is entering a new phase.

For the first time, an experienced engineer can complete substantial migration, maintenance, refactoring, debugging, documentation and development tasks with limited manual production coding.

This does not reduce the importance of programming knowledge. It increases it.

In an AI-assisted workflow, the engineer may write less code directly, but must understand more of the system as a whole. The engineer must define intent, constrain execution, validate results, test behavior, manage risk and accept final responsibility.

This document presents a practical engineering workflow developed through real production work involving legacy systems, software migration, repository analysis, refactoring, debugging, documentation, incremental development, conversational AI, repository-level AI agents, version control, project memory and repository-based documentation.

The goal of this workflow is not to replace software engineering.

The goal is to practice software engineering at a higher level.

AI accelerates development. Responsibility never leaves the engineer.

Table of Contents

- | | |
|---|---|
| 1. Intended Audience | 12. Development Environment |
| 2. Minimal Workflow in One Page | 13. From Rough Intuition to Structured Action |
| 3. The Engineer Remains in Command | 14. Real-World Hybrid Execution |
| 4. A New Reality: Writing Less, Understanding More | 15. Typical Operating Cycle |
| 5. Strategic AI and Operational AI | 16. Prompting as Engineering Control |
| 6. Adaptive Tool Selection | 17. Review as the Core Engineering Activity |
| 7. Project Memory: Giving AI a Stable Context | 18. Testing and Validation |
| 8. Structured Conversational Memory and Conversation Handoffs | 19. Human Responsibility in Professional Contexts |
| 9. Repository Memory and Documentation | 20. What This Workflow Is Not |
| 10. Version Control and Safe Experimentation | 21. Final Principles |
| 11. Security, Privacy and Boundaries | |

1. Intended Audience

This document is intended for software engineers, technical leads, system administrators, developers maintaining legacy applications, and teams introducing AI-assisted development into real projects.

It is not a beginner tutorial on programming, nor a product manual for a specific AI tool. It is a practical workflow for professionals who need to use AI while preserving engineering discipline, traceability, security and responsibility.

The examples refer to tools such as ChatGPT, ChatGPT Projects, Codex, Git and GitHub, but the principles are not tied to a single vendor or product. The workflow is defined by responsibilities, not by tools.

2. Minimal Workflow in One Page

The complete workflow can be summarized in a compact operating cycle:

1. Define the objective.
2. Review the stable project context: requirements, constraints and previous decisions.
3. Check the repository state with Git.
4. Use ChatGPT for reasoning, risk analysis or architectural discussion if needed.
5. Prepare a precise instruction for Codex if repository-level execution is required. ChatGPT can help prepare this instruction, because it can translate the engineer's intention into a precise task description that Codex can execute.
6. Let Codex inspect or modify the repository within bounded constraints.
7. Review the changes with Git.
8. Test the behavior.
9. Document important decisions.
10. Commit accepted changes.
11. Push to GitHub or the chosen remote repository.
12. If the conversation becomes too long, generate a handoff summary before continuing in a new conversation.

The workflow can be reduced to one line:

Define. Contextualize. Constrain. Generate. Review. Test. Document. Commit. Summarize.

This cycle is more important than any individual tool.

3. The Engineer Remains in Command

AI may generate code.

AI may analyze repositories.

AI may suggest architecture.

AI may generate documentation.

AI may accelerate experimentation.

But AI never owns the final decision.

Human judgment remains the final authority.

The engineer defines the intent. The engineer decides the constraints. The engineer validates the result. The engineer accepts responsibility for integration into the real system.

This is the foundation of the workflow.

AI is not treated as an autonomous replacement for engineering responsibility. It is treated as a powerful assistant operating under human direction.

AI may propose. The engineer disposes.

4. A New Reality: Writing Less, Understanding More

In this workflow, the engineer may complete substantial work without manually typing every line of production code.

This does not reduce technical responsibility. It increases it.

The role of the engineer moves upward: from manual production of every instruction to intentional direction, review, validation and system-level understanding.

*You may write none of the code.
You must understand all of the code.*

This principle is especially important in professional environments. AI-generated code is not acceptable merely because it compiles, runs or looks plausible. It must be understood, reviewed and validated.

If the engineer cannot review the generated code, the engineer is not ready to delegate that code.

5. Strategic AI and Operational AI

Not all AI tools play the same role.

A productive AI-assisted workflow distinguishes between strategic AI and operational AI.

Strategic AI

Strategic AI is used for reasoning, clarification and decision-making support.

In this workflow, ChatGPT is used for tasks such as:

- architectural discussion
- debugging hypotheses
- risk analysis
- clarification of rough ideas
- transformation of intuition into engineering decisions
- comparison of possible approaches
- preparation of precise instructions for execution tools such as Codex
- documentation and explanation

ChatGPT is valuable before implementation because it helps structure the problem.

Operational AI

Operational AI is used for repository-level execution.

In this workflow, Codex is used for tasks such as:

- repository mapping
- multi-file analysis
- dependency discovery

- large-scale refactoring
- repository-level code generation
- repetitive migration work
- locating references across the project
- applying bounded changes
- running local checks when appropriate

This creates a hybrid workflow:

*Conversation creates strategy.
Precision creates execution.*

The engineer uses the right AI role for the right stage of the work.

6. Adaptive Tool Selection

AI tools are selected by task, not by habit.

There is no need to force every problem through the same tool or the same interface.

For repository-wide analysis, Codex may be preferred because it can inspect and operate on the codebase directly.

For isolated changes, debugging, architectural discussions, documentation, or single-file analysis, ChatGPT may be enough.

For accepted solutions, manual integration using a local editor such as Sublime Text may be the safest option.

Tool choice depends on:

- scope
- risk
- complexity
- reversibility
- need for repository access
- need for conversation and reasoning
- confidence in the expected change

Use the smallest tool that can safely solve the problem.

This prevents unnecessary automation and keeps the engineer in control.

7. Project Memory: Giving AI a Stable Context

Long engineering projects do not happen in a single session.

Migration, refactoring, debugging and development may continue for weeks or months. During that time, decisions accumulate and context becomes as important as code.

Why was a path chosen?

Why was an alternative rejected?

What constraint was discovered?

What bug was already diagnosed?

What behavior must not be changed?

What requirement is permanent for the project?

If this reasoning is lost, the project becomes harder to maintain and the AI assistant becomes less useful.

For this reason, project memory is part of the workflow.

In this document, “Projects” refers to ChatGPT Projects: persistent working spaces where conversations and uploaded project files can be kept together.

In practice, a ChatGPT Project is used as a persistent working space for a given software project. The project contains not only conversations, but also stable files that define the project context.

These files may include:

- requirements
- constraints
- architectural rules
- migration strategy
- coding conventions
- database notes
- deployment assumptions
- security restrictions
- examples of accepted patterns
- examples of forbidden changes
- summaries of previous phases
- project-specific vocabulary

This is important because the AI assistant should not rediscover the same rules every day. Stable project information should be uploaded once, kept available, and reused across conversations.

*Conversations are for active work.
Project files are for stable context.*

This turns ChatGPT Projects into a form of engineering memory.

They do not replace Git, documentation or tests. They complement them.

Git remembers the code.

Repository documentation remembers implementation decisions close to the system.

ChatGPT Projects remember the working context used during reasoning, planning and AI-assisted execution.

*Git remembers code.
Documentation remembers why.
Projects remember context.*

8. Structured Conversational Memory and Conversation Handoffs

Project memory is only useful if it can be navigated.

A long list of vague conversations is not memory. It is noise.

Conversations should be structured so that future work can resume quickly.

Recommended practices:

- number conversations
- use clear titles
- keep chronological order
- describe the technical focus
- separate unrelated tasks
- create summaries before moving to a new conversation
- preserve important decisions explicitly
- distinguish between facts, assumptions and pending checks

Example structure:

```
01 - Environment Setup
02 - Repository Mapping
03 - Authentication Refactoring
04 - Prompt Design for Codex
05 - Testing and Rollback
06 - Deployment Notes
```

There is also a practical limitation: long conversations become harder to navigate. They may contain too many details, corrections, intermediate attempts and local decisions.

When a conversation becomes too long, the workflow uses a deliberate handoff process.

The engineer asks the AI assistant to generate a precise summary containing all relevant information needed to continue the work in a new conversation.

This handoff summary should include:

- current objective
- completed work
- pending work
- important decisions
- rejected approaches
- files already modified
- files not yet modified
- known bugs
- technical constraints
- commands already executed
- Git branch or repository status if relevant
- conventions that must be preserved
- warnings for the next session

The summary is then pasted into a new conversation inside the same ChatGPT Project. Because the project already contains stable files and the new conversation receives the handoff summary, work can continue with minimal loss of context.

This creates a controlled continuity mechanism:

```
Stable project files → Current conversation → Handoff summary → New conversation
```

The handoff summary is not a casual recap. It is a technical artifact.

Long conversations decay. Good summaries preserve continuity.

A project should not depend on memory by accident. It should preserve memory by design.

9. Repository Memory and Documentation

Conversational memory preserves reasoning.

Repository memory preserves implementation knowledge.

For this reason, the workflow recommends creating a dedicated documentation folder inside the codebase.

Typical examples:

```
/docs  
/documentation  
/engineering_notes
```

The exact name is not important.

The principle is.

Important decisions should live close to the code.

Repository documentation may include:

- architectural decisions
- migration notes
- database changes
- dependency changes
- security considerations
- deployment procedures
- rollback procedures
- testing procedures
- lessons learned
- unresolved technical debt
- known limitations
- environment assumptions

This documentation is versioned together with the source code using Git.

It evolves with the project.

It does not live only in forgotten emails.

It does not live only in personal notebooks.

It lives with the system.

*Code explains what the system does.
Documentation explains why.*

Without documentation, future maintenance becomes archaeology.

10. Version Control and Safe Experimentation

AI accelerates change.

Fast change increases risk.

For this reason, version control becomes more important than ever.

In this workflow, Git is not optional.

It is a safety mechanism.

Before major AI-assisted changes, the current state should be clean, understood and preferably committed.

Experiments should be isolated.

Changes should be reviewed.

Rollback must remain possible.

Basic practices include:

```
git status
```

before starting, and:

```
git diff
```

after AI-generated changes.

Only reviewed and validated changes should be committed.

After validation, changes may be shared through GitHub or an equivalent repository system.

GitHub provides remote history, collaboration, review, traceability and backup.

AI may generate changes quickly.

Version control ensures those changes remain visible, reviewable and reversible.

Speed without reversibility is not engineering.

11. Security, Privacy and Boundaries

AI-assisted development requires clear boundaries.

An AI tool can be extremely useful when it receives enough context to reason about the problem. However, useful context must not become careless exposure.

Professional engineering work may involve credentials, private keys, tokens, production data, personal information, institutional systems, internal URLs, database structures, logs, contracts, security rules or confidential business processes.

Not all of that information should be sent to an AI system.

Before providing context to an AI assistant, the engineer should ask:

- Does the AI need this information to solve the problem?
- Can the information be anonymized?
- Can secrets be removed?
- Can sample data replace real data?
- Can the structure be explained without exposing sensitive content?
- Is this allowed by organizational policy?
- Would this be acceptable if the conversation were later audited?

Examples of information that should normally not be provided directly include:

- passwords
- API keys
- private keys

- access tokens
- production credentials
- full database dumps with real personal data
- confidential documents unrelated to the technical task
- sensitive logs containing user data
- internal secrets embedded in configuration files

This does not mean AI cannot be used in sensitive projects. It means the engineer must control what is shared.

A safer approach is to provide:

- sanitized examples
- reduced reproductions
- anonymized logs
- synthetic data
- schema descriptions without real records
- relevant snippets instead of entire confidential files
- constraints and expected behavior instead of secrets

AI-generated code must also be reviewed for security.

It may introduce or fail to detect risks such as:

- SQL injection
- authentication errors
- authorization bypasses
- insecure file handling
- unsafe logging
- exposure of secrets
- weak validation
- excessive permissions
- fragile error handling

Security review is therefore part of the workflow, not an optional final step.

Provide enough context for reasoning, but never expose secrets unnecessarily.

AI assistance does not reduce security responsibility. It increases the need for deliberate boundaries.

12. Development Environment

This workflow was developed in a hybrid environment based on:

- Microsoft Windows
- Windows Subsystem for Linux
- Ubuntu
- Git
- GitHub
- a lightweight editor such as Sublime Text
- ChatGPT
- Codex

In this setup, Windows provides the desktop environment, graphical tools, browser, editor and hardware compatibility.

WSL and Ubuntu provide a Linux-like development environment suitable for server-side work, command-line tools, scripting, package management, Git operations and repository execution.

Sublime Text is used as a fast local editor for manual inspection and integration.

Codex is used from the terminal when repository-level AI assistance is useful.

These tools are examples.

They are not requirements.

The same workflow may be applied in:

- native Linux environments
- Apple ecosystems using Terminal, Homebrew or Xcode tools
- cloud-based development environments
- containerized environments
- remote development environments

The workflow is defined by responsibilities, not by products.

Tools change. Engineering responsibility does not.

13. From Rough Intuition to Structured Action

One of the most valuable discoveries of this workflow is that conversational AI is not only a code generator.

It is a cognitive amplifier.

In practice, an engineer may begin with:

- an incomplete intuition
- a vague suspicion
- a design concern
- a debugging hypothesis
- an architectural discomfort
- a sense that something is wrong but not yet formalized

Through iterative conversation with ChatGPT, these rough ideas may evolve into:

- precise technical decisions
- implementation strategies
- repository instructions
- risk assessments
- documentation
- executable prompts for Codex
- commit plans
- rollback strategies

Conversation transforms intuition into execution.

This is one of the most important parts of the workflow.

The value of AI is not only that it writes code faster.

The value is that it helps the engineer convert unclear thought into structured action.

14. Real-World Hybrid Execution

In practice, this workflow is not linear.

It is adaptive.

Different situations require different paths.

Scenario 1: Repository mapping

A repository contains hundreds of files. The engineer needs to understand structure, dependencies and execution flow.

A likely path is:

```
Human → Codex → Summary → Human review → Documentation
```

Codex is useful because the task requires repository-level inspection.

Scenario 2: Single-file refactoring

A legacy file needs a controlled compatibility update.

A likely path is:

```
Human → ChatGPT → Manual integration → Git review → Test → Commit
```

ChatGPT may be enough if the file can be analyzed directly and the engineer wants full manual control.

Scenario 3: Risky architectural change

A change affects several modules and may alter behavior.

A likely path is:

```
Human → ChatGPT → Risk analysis → Codex inspection → Git branch → Controlled changes → Tests → Review → Commit
```

The key point is that the engineer chooses the path.

AI never chooses the path.

15. Typical Operating Cycle

A practical operating cycle may look like this:

13. Define the objective.
14. Check the repository state.
15. Review the stable project context: requirements, constraints and previous decisions.
16. Discuss risks and strategy with ChatGPT if needed.
17. Prepare a precise instruction for Codex if repository-level execution is required. ChatGPT can help prepare this instruction, because it can translate the engineer's intention into a precise task description that Codex can execute.
18. Let Codex inspect or modify within bounded constraints.
19. Review changes using Git.
20. Test behavior.
21. Document important decisions.

22. Commit accepted changes.
23. Push to GitHub or the chosen remote repository.
24. If the conversation has become too long, generate a handoff summary before continuing in a new conversation.

This can be reduced to a practical mnemonic:

Define. Contextualize. Constrain. Generate. Review. Test. Document. Commit. Summarize.

The key point is that AI-assisted work is not a single prompt. It is a managed engineering loop.

16. Prompting as Engineering Control

In this workflow, prompting is not treated as casual conversation.

For operational AI tools, prompting becomes a form of engineering control.

A weak instruction is:

Fix this module.

This is too broad. It does not define scope, risk, constraints or expected behavior.

A stronger instruction is:

Inspect this module and identify the files involved in user creation. Do not modify anything yet. Summarize the flow, identify dependencies and point out possible risks.

Another example:

Migrate this file for PHP 8.3 compatibility. Preserve existing behavior. Do not redesign the database access layer. Do not change SQL semantics. Show the diff after the change.

The instruction defines the work boundary.

It tells the AI not only what to do, but what not to do.

Constraints are as important as commands.

17. Review as the Core Engineering Activity

In traditional development, writing code often feels like the central activity.

In AI-assisted development, review becomes even more central.

The engineer must review:

- correctness
- security
- behavior preservation
- architecture
- maintainability
- error handling
- data access
- edge cases
- unintended changes
- consistency with project conventions

AI-generated code may be syntactically correct and still be semantically wrong.

It may solve the visible problem while breaking an invisible constraint.

It may improve style while changing behavior.

For this reason, review cannot be skipped.

Generated code is a proposal, not a conclusion.

18. Testing and Validation

Testing is the point where plausible code meets reality.

AI can produce convincing explanations and elegant implementations, but only execution reveals whether the system behaves correctly.

Validation may include:

- syntax checks
- unit tests
- integration tests
- manual tests
- regression checks
- database verification
- log inspection
- security review
- user workflow testing
- comparison with previous behavior

In legacy systems, validation often means verifying that the new implementation behaves exactly like the old one where behavior must be preserved.

AI can suggest truth. Tests expose truth.

19. Human Responsibility in Professional Contexts

In professional environments, the responsibility question cannot be avoided.

If a generated change breaks production, exposes data, corrupts records or alters business behavior, the explanation cannot be:

The AI wrote it.

That is not an engineering answer.

The accepted change belongs to the engineer and the organization that approved it.

This is why review, version control, documentation and testing are not bureaucratic additions.

They are what make AI-assisted development professionally usable.

Delegation to AI does not remove accountability. It increases the need for control.

20. What This Workflow Is Not

This workflow is not blind automation.

It is not prompt-and-paste programming.

It is not a replacement for Git.

It is not a replacement for testing.

It is not a replacement for architecture.

It is not a replacement for documentation.

It is not a justification for accepting code that nobody understands.

This workflow is a disciplined way to combine human engineering judgment with AI acceleration.

21. Final Principles

The workflow may be summarized as:

```
Human defines.  
Project files contextualize.  
AI assists.  
Conversations evolve.  
Summaries preserve continuity.  
Git protects.  
GitHub shares.  
Documentation explains.  
Tests validate.  
Responsibility remains human.
```

And in an even shorter form:

*You may write none of the code.
You must understand all of the code.*

Another essential principle is:

*Do not rely on accidental memory.
Build memory into the workflow.*

The final principle remains:

*AI accelerates development.
Responsibility never leaves the engineer.*

This is the essence of the workflow.